Laboratory Guide 4: Variational Quantum Algorithms

Hybrid Classical-Quantum Computing: VQE and QAOA

Course: Quantum Computing Foundation

Contents

1 Introduction and Objectives **2**

2 Part A: The Hybrid Paradigm **2**
2.1 Architecture of the Variational Loop 2
2.2 The Ansatz: Expressibility vs. Trainability 3

3 Part B: The Parameter-Shift Rule **3**
3.1 The Gradient Problem in Quantum Circuits 3
3.2 Derivation 3
3.3 Concrete Example 4
3.4 Barren Plateaus 5

4 Part C: VQE — The Variational Quantum Eigensolver **5**
4.1 Goal and Formulation 5
4.2 Case Study: 2-Qubit Transverse Ising Model 5
4.3 Hardware-Efficient Ansatz for H 5
4.4 Gradient Computation via Parameter-Shift 6

5 Part D: QAOA — Quantum Approximate Optimization **6**
5.1 From VQE to Combinatorial Optimization 6
5.2 Case Study: MaxCut on a 4-Node Cycle 6
5.3 Approximation Ratio and the Role of p 7

6 Part E: IBM Composer Exercises **7**
6.1 Exercise 1: The Parameter-Shift Rule in Practice 8
6.2 Exercise 2: Visualise the Cost Landscape 8

7 Part F: Qiskit Python Implementation **8**

8 Part G: Bridge to Lab 5 and Assessment **9**
8.1 Assessment 9

1 Introduction and Objectives

Labs 3, 3.5 and 3.6 introduced the foundations of quantum interference, search, and integer factoring. All those algorithms assumed a *fault-tolerant* quantum computer: a device with essentially unlimited coherence time and negligible gate error. Today's hardware — the so-called NISQ era — does not meet this assumption. Circuits deeper than a few dozen layers are dominated by noise before they can return a useful result.

This laboratory introduces the practical answer to the NISQ limitation: **variational quantum algorithms**, also called *hybrid classical-quantum* algorithms. Rather than running a single deep quantum circuit, these algorithms iterate between a short parametric quantum circuit and a classical optimizer, converging toward a useful solution. Understanding this paradigm is a prerequisite for Lab 5, where you will apply it to a real-world portfolio optimization problem.

Prerequisites:

- Lab 3.6 (Shor's Algorithm) and all preceding labs
- Basic linear algebra: eigenvalues, Hermitian matrices, tensor products
- Classical optimization concepts: gradient descent, cost functions

Learning Objectives:

1. Understand why the NISQ era requires a hybrid paradigm.
2. Master parametric quantum circuits (PQC) and the concept of ansatz.
3. Derive and apply the *parameter-shift rule* for quantum gradients.
4. Implement the Variational Quantum Eigensolver (VQE) for a 2-qubit Ising model.
5. Implement QAOA for MaxCut and understand the QUBO framework that bridges directly to financial optimization.

The NISQ Bottleneck

Current quantum computers (IBM Eagle, Heron; Google Sycamore; IonQ Forte) have:

- **Gate errors:** $\sim$0.1–1% per two-qubit gate.
- **Coherence times:** $T_1, T_2 \sim 100$–$500\,\mu$s.
- **Depth limit:** circuits deeper than $\sim$50–200 gates produce results dominated by noise.

Shor's Algorithm for RSA-2048 requires $\sim 10^9$ fault-tolerant gates and millions of error-corrected qubits. VQE and QAOA are designed to run on circuits with $\mathcal{O}(n)$–$\mathcal{O}(n^2)$ gates and can *tolerate some noise* because the classical optimizer can compensate for small systematic errors.

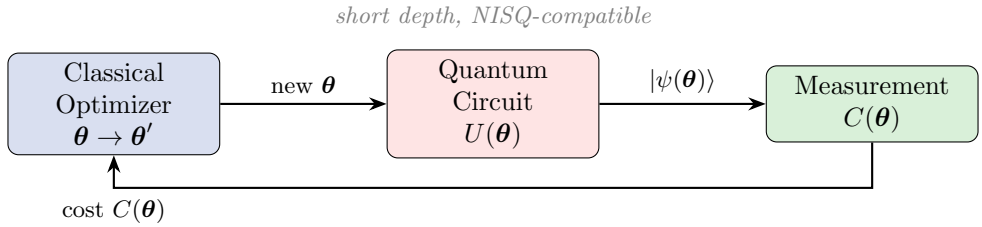
2 Part A: The Hybrid Paradigm

2.1 Architecture of the Variational Loop

A variational quantum algorithm operates as follows:

1. **Encode:** Map the problem into a Hamiltonian H such that the solution corresponds to the ground state (lowest eigenvalue) of H.

2. **Ansatz:** Choose a parametric quantum circuit $U(\boldsymbol{\theta})$ — the *ansatz* — that produces the trial state $|\psi(\boldsymbol{\theta})\rangle = U(\boldsymbol{\theta})\,|0\rangle$.
3. **Measure:** Evaluate the cost function $C(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|\,H\,|\psi(\boldsymbol{\theta})\rangle = \langle H\rangle_{\boldsymbol{\theta}}$.
4. **Optimise:** A *classical* optimizer updates $\boldsymbol{\theta} \to \boldsymbol{\theta}'$ to reduce C.
5. **Repeat:** Steps 3–4 iterate until convergence.



2.2 The Ansatz: Expressibility vs. Trainability

The ansatz $U(\boldsymbol{\theta})$ must satisfy two competing requirements:

- **Expressibility:** the set of states $\{|\psi(\boldsymbol{\theta})\rangle\}$ should include (or approximate) the true ground state.
- **Trainability:** gradients $\partial C/\partial\theta_k$ must be non-negligible so the optimizer can navigate the landscape.

Hardware-Efficient Ansatz (HEA)

An ansatz built from gates native to the target hardware: alternating layers of single-qubit rotations $R_y(\theta)$, $R_z(\phi)$ and entangling CNOT gates. For n qubits and L layers, the circuit has $\mathcal{O}(nL)$ parameters. Maximises expressibility within the coherence budget, but may not respect problem symmetry.

3 Part B: The Parameter-Shift Rule

3.1 The Gradient Problem in Quantum Circuits

Classical neural networks use backpropagation to compute gradients exactly in one backward pass. In a quantum circuit, however, we cannot backpropagate through a unitary — we can only *evaluate* the circuit at different parameter settings. The **parameter-shift rule** (Mitarai et al., 2018; Schuld et al., 2019) gives us an exact gradient using only two additional circuit evaluations.

3.2 Derivation

Consider a gate of the form $G(\theta) = e^{-i\theta P/2}$, where P is a Hermitian operator with eigenvalues ± 1 (a Pauli operator: $P \in \{X, Y, Z, X \otimes Z, \ldots\}$). Then:

$$G(\theta) = e^{-i\theta P/2} = \cos\tfrac{\theta}{2}\, I - i \sin\tfrac{\theta}{2}\, P. \tag{1}$$

The cost function (measuring observable O after applying a circuit $V(\theta) = \cdots G(\theta) \cdots$ to state ρ) is:

$$f(\theta) = \mathrm{Tr}\Big[O\, V(\theta)\, \rho\, V^\dagger(\theta)\Big]. \tag{2}$$

Parameter-Shift Rule — Full Derivation

Step 1: Expand $G(\theta)$. Using $\cos(\theta/2) = \frac{e^{i\theta/2}+e^{-i\theta/2}}{2}$ and $\sin(\theta/2) = \frac{e^{i\theta/2}-e^{-i\theta/2}}{2i}$, and denoting the context of the circuit (all gates before and after G) as a fixed superoperator $\mathcal{L}$, we can write:

$$f(\theta) = \mathrm{Tr}\left[\tilde{O}(\theta)\right] = a + b\cos\theta + c\sin\theta \tag{3}$$

where $a, b, c \in \mathbb{R}$ depend on O, ρ and the rest of the circuit, but *not* on θ. This is proved by expanding:

$$\begin{aligned} V(\theta)\rho V^\dagger(\theta) &= \left(\cos\tfrac{\theta}{2}\, I - i\sin\tfrac{\theta}{2}\, P\right)\tilde{\rho}\left(\cos\tfrac{\theta}{2}\, I + i\sin\tfrac{\theta}{2}\, P\right) \\ &= \cos^2\tfrac{\theta}{2}\,\tilde{\rho} + \sin^2\tfrac{\theta}{2}\, P\tilde{\rho}P + i\sin\tfrac{\theta}{2}\cos\tfrac{\theta}{2}\,[\tilde{\rho}, P], \end{aligned}$$

where $\tilde{\rho}$ absorbs all gates not containing θ. Applying $\mathrm{Tr}[O\,\cdot\,]$ and using $\cos^2 = (1+\cos\theta)/2$, $\sin^2 = (1-\cos\theta)/2$, $\sin(\theta/2)\cos(\theta/2) = \sin\theta/2$:

$$\begin{aligned} f(\theta) &= \frac{1+\cos\theta}{2}\,\mathrm{Tr}[O\tilde{\rho}] + \frac{1-\cos\theta}{2}\,\mathrm{Tr}[OP\tilde{\rho}P] + \frac{\sin\theta}{2}\,\underbrace{i\,\mathrm{Tr}[O[\tilde{\rho}, P]]}_{\in\mathbb{R}} \\ &= \underbrace{\tfrac{1}{2}\big(\mathrm{Tr}[O\tilde{\rho}] + \mathrm{Tr}[OP\tilde{\rho}P]\big)}_{a} + \underbrace{\tfrac{1}{2}\big(\mathrm{Tr}[O\tilde{\rho}] - \mathrm{Tr}[OP\tilde{\rho}P]\big)}_{b}\cos\theta + c\sin\theta. \end{aligned}$$

Step 2: Differentiate the trigonometric form.
Since $f(\theta) = a + b\cos\theta + c\sin\theta$:

$$\frac{df}{d\theta} = -b\sin\theta + c\cos\theta. \tag{4}$$

Step 3: Express the derivative via two circuit evaluations.
Evaluate f at the shifted points:

$$f\left(\theta + \tfrac{\pi}{2}\right) = a + b\cos\left(\theta + \tfrac{\pi}{2}\right) + c\sin\left(\theta + \tfrac{\pi}{2}\right) = a - b\sin\theta + c\cos\theta, \tag{5}$$

$$f\left(\theta - \tfrac{\pi}{2}\right) = a + b\cos\left(\theta - \tfrac{\pi}{2}\right) + c\sin\left(\theta - \tfrac{\pi}{2}\right) = a + b\sin\theta - c\cos\theta. \tag{6}$$

Subtracting:

$$f\left(\theta + \tfrac{\pi}{2}\right) - f\left(\theta - \tfrac{\pi}{2}\right) = -2b\sin\theta + 2c\cos\theta = 2\,\frac{df}{d\theta}. \tag{7}$$

Result:

$$\boxed{\frac{\partial f}{\partial \theta_k} = \frac{1}{2}\left[f\left(\theta_k + \frac{\pi}{2}\right) - f\left(\theta_k - \frac{\pi}{2}\right)\right].} \tag{8}$$

This is **exact**, not an approximation. The gradient of any expectation value with respect to a Pauli-generator rotation gate requires exactly **two additional circuit evaluations**.

3.3 Concrete Example

For the circuit $R_y(\theta)\,|0\rangle$ measuring $\langle Z\rangle$:

$$f(\theta) = \langle 0|\, R_y^\dagger(\theta)\, Z\, R_y(\theta)\,|0\rangle = \cos^2\tfrac{\theta}{2} - \sin^2\tfrac{\theta}{2} = \cos\theta.$$

Applying equation (8):

$$\frac{df}{d\theta} = \frac{f(\theta+\pi/2) - f(\theta-\pi/2)}{2} = \frac{\cos(\theta+\pi/2) - \cos(\theta-\pi/2)}{2} = \frac{-\sin\theta - \sin\theta}{2} = -\sin\theta. \quad \checkmark$$

3.4 Barren Plateaus

The parameter-shift rule guarantees we can *compute* gradients, but not that gradients are *large*. As the number of qubits n grows, random parametric circuits suffer the **barren plateau** phenomenon:

Barren Plateaus (McClean et al.

For a random n-qubit ansatz, the variance of the gradient $\partial C/\partial\theta_k$ decreases exponentially:

$$\mathrm{Var}\left[\frac{\partial C}{\partial \theta_k}\right] = \mathcal{O}\left(\frac{1}{2^n}\right).$$

This means gradients are exponentially small in expectation — the optimizer is lost on a flat landscape. Mitigations include: problem-inspired ansätze, layerwise training, natural gradient methods, and classical pre-training.

4 Part C: VQE — The Variational Quantum Eigensolver

4.1 Goal and Formulation

VQE (Peruzzo et al., 2014) finds the ground-state energy of a Hamiltonian H by exploiting the *variational principle* of quantum mechanics:

$$E_0 \;\leq\; C(\boldsymbol{\theta}) = \langle\psi(\boldsymbol{\theta})|\; H \;|\psi(\boldsymbol{\theta})\rangle \quad \forall\,\boldsymbol{\theta}. \tag{9}$$

The optimizer minimises $C(\boldsymbol{\theta})$ over $\boldsymbol{\theta}$, pushing the trial state toward the ground state.

4.2 Case Study: 2-Qubit Transverse Ising Model

We work with the Hamiltonian:

$$H = Z_0 Z_1 + 0.5\, Z_0, \tag{10}$$

where Z_i denotes the Pauli Z operator on qubit i. The $Z_0 Z_1$ term penalises states where the two qubits are anti-aligned; $0.5\, Z_0$ introduces a longitudinal field on qubit 0.

Ground State of Equation (10)

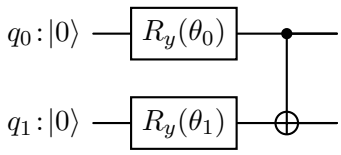
The matrix of H in the $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$ basis is diagonal (since H is a sum of Pauli-Z products):

$$H = \mathrm{diag}(1+0.5,\; -1+0.5,\; -1-0.5,\; 1-0.5) = \mathrm{diag}(1.5,\; -0.5,\; -1.5,\; 0.5).$$

- Ground state: $|10\rangle$ with energy $E_0 = -1.5$.
- First excited: $|01\rangle$ with energy -0.5.

4.3 Hardware-Efficient Ansatz for H

We use a 2-parameter ansatz:



$$|\psi(\theta_0, \theta_1)\rangle = \mathrm{CX}_{01} \left(R_y(\theta_0) \otimes R_y(\theta_1)\right) |00\rangle \,.$$

The cost function:

$$C(\theta_0, \theta_1) = \langle\psi(\theta_0, \theta_1)| \left(Z_0 Z_1 + 0.5\, Z_0\right) |\psi(\theta_0, \theta_1)\rangle \,. \tag{11}$$

Student Activity

Verify: Set $\theta_0 = \pi$, $\theta_1 = 0$. Show that $|\psi(\pi, 0)\rangle = |10\rangle$ and that $C(\pi, 0) = -1.5 = E_0$. *Hint:* $R_y(\pi)\,|0\rangle = |1\rangle$, $R_y(0)\,|0\rangle = |0\rangle$, and CX leaves $|10\rangle$ unchanged.

4.4 Gradient Computation via Parameter-Shift

For θ_0:

$$\frac{\partial C}{\partial \theta_0} = \frac{1}{2}\big[C(\theta_0 + \tfrac{\pi}{2}, \theta_1) - C(\theta_0 - \tfrac{\pi}{2}, \theta_1)\big]. \tag{12}$$

The optimizer (COBYLA or L-BFGS-B) uses these gradient estimates to descend toward the minimum $(\pi, 0)$.

5 Part D: QAOA — Quantum Approximate Optimization

5.1 From VQE to Combinatorial Optimization

VQE can be applied to *any* Hamiltonian whose ground state encodes the optimal solution to a problem. For *combinatorial* problems over binary variables $x \in \{0, 1\}^n$, the standard encoding uses:

$$z_i = 1 - 2x_i \in \{-1, +1\} \quad \Longleftrightarrow \quad x_i = \frac{1 - Z_i}{2}. \tag{13}$$

Any quadratic binary objective function $\min_x \sum_{ij} Q_{ij} x_i x_j + \sum_i q_i x_i$ maps to a Hamiltonian that is a sum of Pauli-Z products.

QAOA (Farhi et al., 2014) is a VQE with a *structured* ansatz inspired by adiabatic quantum computing. It alternates two unitaries for p layers:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = e^{-i\beta_p H_B}\, e^{-i\gamma_p H_C} \,\cdots\, e^{-i\beta_1 H_B}\, e^{-i\gamma_1 H_C}\, |+\rangle^{\otimes n}\,, \tag{14}$$

where:

- H_C is the **cost Hamiltonian** encoding the problem.
- $H_B = \sum_i X_i$ is the **mixer Hamiltonian** promoting transitions between solutions.
- $(\boldsymbol{\gamma}, \boldsymbol{\beta}) \in \mathbb{R}^{2p}$ are optimised classically.

As $p \to \infty$, QAOA converges to the exact solution.

5.2 Case Study: MaxCut on a 4-Node Cycle

MaxCut

Given a graph $G = (V, E)$, partition V into two sets S and $\bar{S}$ to maximise the number of edges crossing the partition (i.e. edges with one endpoint in S and one in $\bar{S}$).

We use the 4-node cycle C_4 with edges $E = \{(0, 1), (1, 2), (2, 3), (3, 0)\}$ (see Figure 1).

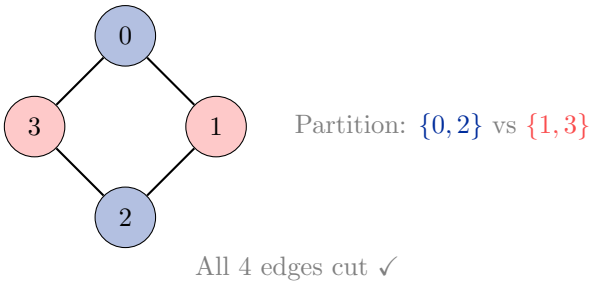


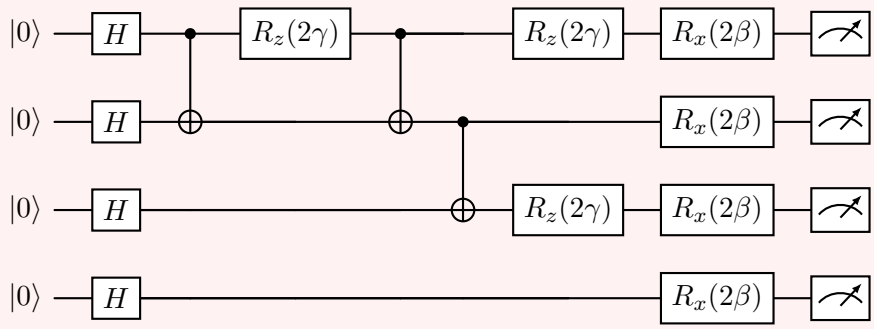
Figure 1: 4-node cycle C_4 with the optimal MaxCut partition.

The cost Hamiltonian encoding the number of cut edges:

$$\begin{aligned} H_C &= \frac{1}{2} \sum_{(i,j)\in E} \big(I - Z_i Z_j\big) \\ &= 2\,I - \frac{1}{2}\big(Z_0Z_1 + Z_1Z_2 + Z_2Z_3 + Z_3Z_0\big). \end{aligned} \tag{15}$$

The maximum cut value is 4, achieved by $|0101\rangle$ and $|1010\rangle$ (alternating-spin states, corresponding to the partition $\{0,2\}$ vs $\{1,3\}$).

QAOA Circuit — p



The cost unitary $e^{-i\gamma H_C}$ decomposes as: for each edge (i,j): $\texttt{CNOT}(i,j)$ — $R_z(2\gamma)$ on j — $\texttt{CNOT}(i,j)$. This implements $e^{-i\gamma Z_i Z_j}$ via:

$$e^{-i\gamma Z_i Z_j} = \mathrm{CNOT}_{ij} \cdot \big(I \otimes e^{-i\gamma Z_j}\big) \cdot \mathrm{CNOT}_{ij}.$$

The mixer $e^{-i\beta H_B} = \bigotimes_i R_x(2\beta)$ applies a global X-rotation to all qubits.

5.3 Approximation Ratio and the Role of p

Define the **approximation ratio**: $r = \langle H_C \rangle / \mathrm{MaxCut}$. For C_4 with $p = 1$, the optimal ratio is $r \approx 0.75$ (3 expected cuts out of 4). Increasing p improves r:

p	Parameters	Approx. ratio (simulation)
1	2	≈ 0.75
2	4	≈ 0.94
3	6	≈ 0.99

6 Part E: IBM Composer Exercises

Platform: IBM Quantum Composer — `https://quantum.ibm.com/composer`

6.1 Exercise 1: The Parameter-Shift Rule in Practice

Student Activity

Observe the parameter-shift rule directly:

1. Build the 1-qubit circuit: $R_y(\theta)\,|0\rangle$, measure $\langle Z\rangle$.
2. Set $\theta = 0.5$ rad. Record $f(0.5)$.
3. Change to $\theta = 0.5 + \pi/2$. Record f^+.
4. Change to $\theta = 0.5 - \pi/2$. Record f^-.
5. Compute $(f^+ - f^-)/2$. Compare with the analytic $-\sin(0.5) \approx -0.479$.

Expected: The parameter-shift estimate matches the analytic gradient exactly (within shot noise).

6.2 Exercise 2: Visualise the Cost Landscape

Student Activity

Two-qubit VQE landscape:

1. Build the ansatz $\mathrm{CX}_{01}(R_y(\theta_0) \otimes R_y(\theta_1))$.
2. Fix $\theta_1 = 0$. Sweep $\theta_0 \in \{0, \pi/4, \pi/2, 3\pi/4, \pi\}$.
3. For each value, add Z_0Z_1 and Z_0 measurement and record $C(\theta_0, 0)$ from the Probabilities panel.
4. Plot by hand: you should see C decrease monotonically from 1.5 (at $\theta_0 = 0$) to -1.5 (at $\theta_0 = \pi$).

7 Part F: Qiskit Python Implementation

The full implementation is in the companion Jupyter notebook. Three exercises are provided:

1. **Parameter-shift demo:** compute the gradient of $\langle Z\rangle_\theta$ analytically and via the rule, verify agreement.
2. **VQE on Ising:** use Qiskit's `Estimator` primitive with COBYLA to minimise $C(\theta_0, \theta_1)$.
3. **QAOA MaxCut:** sweep $p \in \{1, 2, 3\}$ and compare approximation ratios and convergence speed.

Key Code Structure

```python
# VQE cost function (Estimator-based)
def cost_function(params, estimator, circuit, hamiltonian):
    job    = estimator.run([(circuit, hamiltonian, params)])
    result = job.result()
    return result[0].data.evs   # expectation value

# QAOA circuit for MaxCut (one p-layer)
def build_qaoa(n, edges, p):
    gammas = [Parameter(f'g{i}') for i in range(p)]
    betas  = [Parameter(f'b{i}') for i in range(p)]
    qc     = QuantumCircuit(n)
    qc.h(range(n))
    for k in range(p):
        for (u, v) in edges:        # cost unitary
            qc.cx(u, v)
            qc.rz(2 * gammas[k], v)
            qc.cx(u, v)
        qc.rx(2 * betas[k], range(n))   # mixer
    return qc, gammas + betas
```

8 Part G: Bridge to Lab 5 and Assessment

Bridge to Lab 5: Finance & Industry

From MaxCut to Portfolio Optimization
In Lab 5, the portfolio optimization problem is formulated as:

$$\min_{x \in \{0,1\}^n} x^T Q x \quad \text{subject to} \quad \mathbf{1}^T x = K,$$

where $x_i = 1$ means "invest in asset i", K is the target portfolio size, and Q_{ij} encodes the risk-return trade-off (covariance minus expected return).
This is a **QUBO** (Quadratic Unconstrained Binary Optimization) and maps to a Hamiltonian via $Z_i = 1 - 2x_i$:

$$H_{\text{portfolio}} = \sum_{i,j} Q_{ij} \frac{(I - Z_i)(I - Z_j)}{4} + \lambda \left(\sum_i \frac{I - Z_i}{2} - K \right)^2 .$$

The second term is a *penalty* enforcing the budget constraint. QAOA is then applied to $H_{\text{portfolio}}$ using *exactly the same circuit structure* built in this lab, but with this financial Hamiltonian as H_C.
The skills you need: variational loop (§**??**), parameter-shift gradients (§**??**), QAOA circuit construction (§**??**).

8.1 Assessment

Q1. Parameter-Shift Rule
Why does the parameter-shift rule require *exactly two* additional circuit evaluations per parameter, regardless of the circuit depth?

A) Because the gradient is approximated by a finite difference.

B) Because $f(\theta)$ is a trigonometric polynomial of degree 1 in θ (for Pauli-generator gates), and its derivative is uniquely determined by evaluations at $\theta \pm \pi/2$.

C) Because quantum circuits have only one parameter each.

Q2. Barren Plateaus

Which strategy does *not* mitigate the barren plateau problem?

A) Using problem-inspired ansätze that respect problem symmetry.

B) Increasing the number of qubits to improve expressibility.

C) Layerwise training, initialising one layer at a time.

Q3. QAOA Structure

In QAOA, what is the role of the mixer Hamiltonian $H_B = \sum_i X_i$?

A) To encode the cost function directly into the circuit.

B) To create transitions between different bitstring states, allowing the optimizer to escape local minima.

C) To initialise all qubits in the computational basis.

Q4. QUBO Connection

A portfolio with $n = 4$ assets uses a QUBO objective $x^T Q x$. After substituting $x_i = (1 - Z_i)/2$, the resulting Hamiltonian H_C contains terms of the form $Z_i Z_j$. What gate in the QAOA circuit directly implements $e^{-i\gamma Z_i Z_j}$?

A) A controlled-Z gate.

B) CNOT — $R_z(2\gamma)$ — CNOT on qubits i and j.

C) Two $R_z(2\gamma)$ gates applied independently.

Solutions: Q1: B (the trigonometric form $a + b\cos\theta + c\sin\theta$ is fundamental — any expectation value w.r.t. a Pauli generator is of this form, making the two-point shift exact, not approximate); Q2: B (more qubits make barren plateaus *worse*, not better); Q3: B (the mixer drives transitions, analogous to kinetic energy in the adiabatic picture — without it the circuit is stuck in the product state and cannot explore the solution space); Q4: B (the standard ZZ-rotation decomposition: $e^{-i\gamma Z_i Z_j} = \mathrm{CNOT}_{ij} \cdot (I \otimes e^{-i\gamma Z_j}) \cdot \mathrm{CNOT}_{ij}$, with $e^{-i\gamma Z} = R_z(2\gamma)$ up to global phase).